

Week 4

Game Engine Development Unity Audio

Hooman Salamat

Objective & Concept

Objective

- Unity's audio features include full 3D spatial sound, real-time mixing and mastering, hierarchies of mixers, snapshots, and predefined effects. Read this section to learn about audio in Unity, including clips, sources, listeners, importing, and sound settings.

Concepts

- A game would be incomplete without some kind of audio, be it background music or sound effects. Unity's audio system can import most standard audio file formats, play sounds in 3D space, and apply optional effects such as echo and filtering. Unity can also record audio from any available microphone on your machine for use during gameplay or for storage and transmission.

Audio overview

In real life, objects emit sounds that listeners hear.

The way a sound is perceived depends on many factors.

A listener can tell roughly which direction a sound is coming from and may also get some sense of its distance from its loudness and quality.

A fast-moving sound source (such as a falling bomb or a passing police car) changes in pitch as it moves as a result of the Doppler Effect.

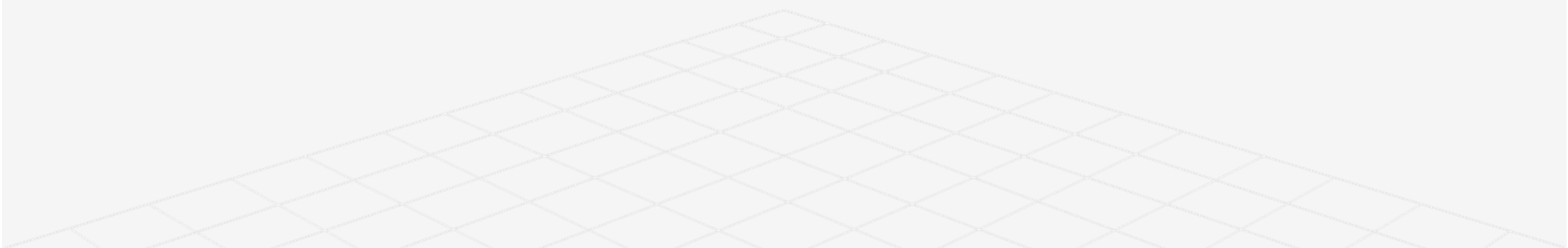
Surroundings also affect the way sound is reflected. A voice inside a cave has an echo, but the same voice in the open air doesn't.

To simulate the effects of position, Unity requires sounds to originate from Audio Sources attached to objects.

The sounds emitted are then picked up by an Audio Listener attached to another object, most often the main camera.

Unity can then simulate the effects of a source's distance and position from the listener object and play them to you accordingly.

You can also use the relative speed of the source and listener objects to simulate the Doppler Effect for added realism.



Basic theory

Unity can't calculate echoes purely from scene geometry, but you can simulate them by adding Audio Filters to objects.

For example, you could apply the Echo filter to a sound that is supposed to be coming from inside a cave.

In situations where objects can move in and out of a place with a strong echo, you can add a Reverb Zone to the scene.

For example, your game might involve cars driving through a tunnel. If you place a reverb zone inside the tunnel, the cars' engine sounds start to echo as they enter. The echo quiets as the cars emerge from the other side.

With the Unity Audio Mixer, you can mix various audio sources, apply effects to them, and perform mastering.



Working with audio assets

Unity can import audio files in **AIFF**, **WAV**, **MP3**, and **Ogg** formats in the same way as other assets.

Drag the files into the Project panel.

Import an audio file to create an Audio Clip that you can then drag to an Audio Source or use from a script.

The Audio Clip reference page has more details about the import options available for audio files.

For music, Unity also supports tracker modules, which use short audio samples as “instruments” that you can arrange to play tunes.

You can import tracker modules from **.xm**, **.mod**, **.it**, and **.s3m** files and use them the same way you use other audio clips.

Audio recording

Unity can access the computer's microphones from a script and create Audio Clips by direct recording.

The Microphone class provides an API to find available microphones, query their capabilities, and start and end a recording session.

The script reference page for [Microphone](#) has further information and code samples for audio recording.

Introduction to audio files

As with Meshes or Textures, the workflow for **Audio File** assets is designed to be smooth and simple. Unity can import almost every common file format, but there are a few details that are useful to be aware of when you work with audio files.

For an extensive description of the compression formats and other options available for encoding audio data, refer to [Audio Clip](#)

Audio data and Audio Clips

Unity audio data is separate from the actual Audio Clips. Audio data is the raw information about an audio file. It contains information such as length, channel count, sample rate, and compression format.

Audio Clips are assets that contain the audio data and processes the audio data for use with Unity Engine. Audio Clips contain the audio data used by [Audio Sources](#)

Alter the settings of your audio file

Use the settings in the [Audio Clip Import Settings](#) to determine how the clips will load at runtime. These settings let you decide which audio assets will stay in memory. This is ideal for frequent or unpredictable sounds like footsteps, weapons, or impacts.

For other assets, such as speech, background music, or ambient loops, you can set them to load on-demand or as the player progresses. This approach optimizes memory usage and improves performance.

Use audio files in your scripts

Any audio file imported into Unity is available from scripts as an AudioClip instance. This provides a way for the game runtime of the audio system to access the encoded audio data. The game might access meta-information about the audio data via the AudioClip even before the actual audio data loads.

The import process extracts information such as length, channel count and sample rate from the encoded audio data and stores these details in the AudioClip. This information can be useful for automatic dialog or music sequencing systems, because the music engine can use the length to schedule music playback before the data loads. It also helps to reduce memory usage because it only keeps the audio clips in memory the application needs at a certain time.

Supported Audio file formats

Unity supports mono, stereo, and multichannel audio assets up to eight channels.

You can import the following audio file formats into Unity:

Format	Extensions
MPEG layer 3	.mp3
Audio Interchange File Format	.aiff / .aif
Microsoft Wave	.wav
Ogg Vorbis	.ogg
Free Lossless Audio Codec (FLAC)	.flac

For information about how Unity encodes your files once they are imported, refer to [Audio file compression in Unity](#).

Unity also lets you import [tracker modules](#) in the following formats:

Format	Extensions
Ultimate Soundtracker module	.mod
FastTracker 2 module	.xm
Impulse Tracker module	.it
Scream Tracker module	.s3m

Audio file compression in Unity

Audio file compression is an essential way to optimize your application's performance. When you [import an audio file](#), Unity transcodes the file to a format suitable for the build target and the type of sound.

This page gives an overview of some of the different compression formats and their impact on file size, performance, and audio quality.

To change the compression format of an audio file, use the **Compression Format** setting in the [Audio Clip Import Settings](#).

Audio file compression format types

When Unity encodes audio, the main compression formats for how Unity stores the audio on disk are the following:

PCM

ADPCM

Compressed

PCM or Vorbis/MP3

PCM and Vorbis/MP3 formats keep audio as close as possible to the original sound.

PCM is an uncompressed compression format. PCM is easy for the CPU to handle because the sound is uncompressed, so it plays directly from memory without extra processing.

The Vorbis or MP3 formats are compressed formats. These formats let you use the **Quality** slider to adjust the compression, which removes less noticeable audio to save space but keeps the sound as clear as possible.

ADPCM

ADPCM uses slightly more CPU than the uncompressed PCM option, but yields a constant 3.5 compression factor, which is usually about 3 times worse than the compression that can be achieved with Vorbis or MP3 compression. Like PCM, ADPCM lets you automatically optimize or manually adjust the sample rate to further reduce file size, depending on the sound's frequency content and the acceptable loss of quality.

ADPCM is less suitable for smooth signals like music or ambient sounds because it introduces noticeable artefacts that affect audio quality.

Module files

Module files (.mod,.it,.s3m,.xm) can deliver high quality sounds with a low footprint. When you use module files, make sure that the **Load Type** is set to **Compressed In Memory** in the Audio Clip Import Settings. If it's set to **Decompress On Load**, the whole song will be decompressed.

Compressed audio

Compressed audio (or modules) are best for long files like background music or dialog. Use the **Quality** slider to change the amount of compression. Start with high-quality compression and gradually reduce the setting to the point where the loss of sound quality is perceptible. Then, increase it again slightly until the perceived loss of quality disappears.

Scriptable audio pipeline

The scriptable audio pipeline framework lets you extend the Unity audio engine at specific integration points using Burst-compatible HPC#.

Topic	Description
Scriptable processors concepts	Learn the core concepts required to work with scriptable processors.
Generators	Learn about generators.
Root outputs	Learn about root outputs.
Example: create a generator	Create a generator and attach it to an audio source.
Example: create a root output	Create a root output and connect it to the Unity audio engine.

Tracker Modules

Tracker Modules are essentially just packages of audio samples that have been modeled, arranged and sequenced programatically.

The concept was introduced in the 1980's (mainly in conjunction with the Amiga computer) and has been popular since the early days of game development and demo culture.

Tracker Module files are similar to MIDI files in many ways.

The tracks are scores that contain information about when to play the instruments, and at what pitch and volume and from this, the melody and rhythm of the original tune can be recreated.

However, MIDI has a disadvantage in that the sounds are dependent on the sound bank available in the audio hardware, so MIDI music can sound different on different computers.

In contrast, tracker modules include high quality PCM samples that ensure a similar experience regardless of the audio hardware in use.

Supported formats

Unity supports the four most common module file formats, namely Impulse Tracker (**.it**), Scream Tracker (**.s3m**), Extended Module File Format (**.xm**), and the original Module File Format (**.mod**).

Benefits of Using Tracker Modules

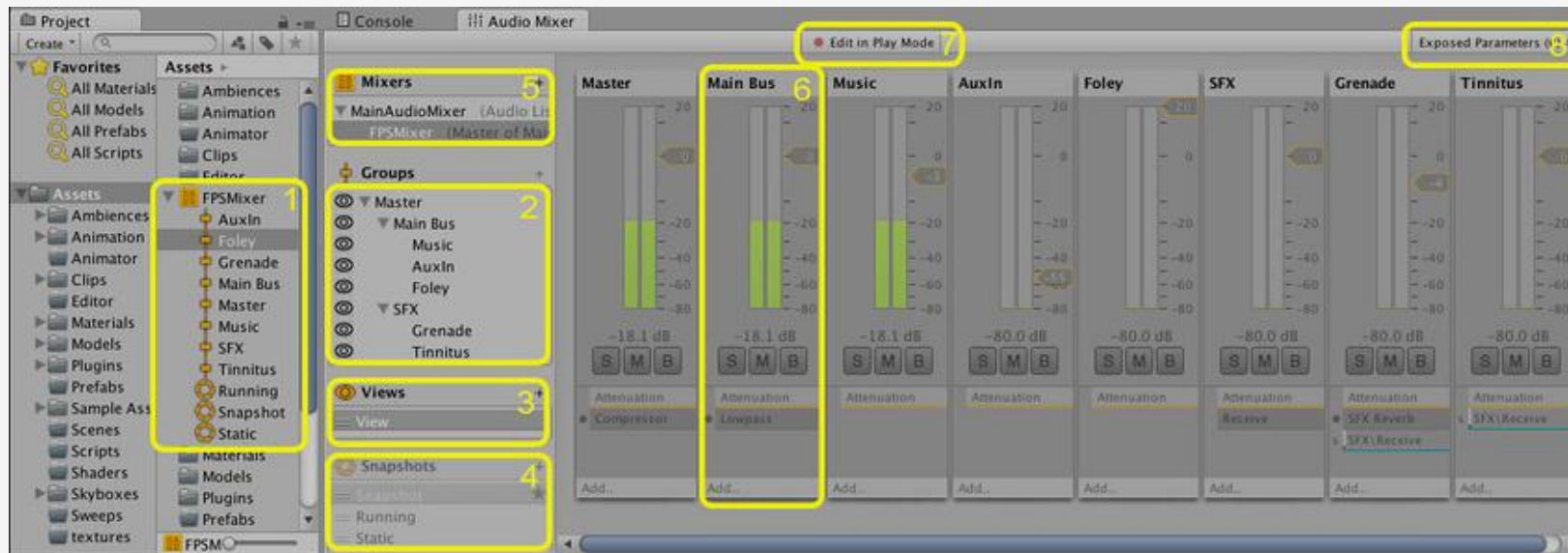
Tracker module files differ from mainstream PCM formats (**.aif**, **.wav**, **.mp3**, and **.ogg**) in that they can be very small without a corresponding loss of sound quality. A single sound sample can be modified in pitch and volume (and can have other effects applied), so it essentially acts as an “instrument” which can play a tune without the overhead of recording the whole tune as a sample. As a result, tracker modules lend themselves to games, where music is required but where a large file download would be a problem.

Audio Mixer

The Audio Mixer is an asset that Audio Sources reference to apply complex routing and mixing to the audio signal they generate.

The Audio Mixer performs this category-based mixing through a hierarchy of groups that you construct inside the asset.

You can apply digital signal processing (DSP) effects and other audio mastering to the signal as Unity routes it from the Audio Source to the Audio Listener.



Audio Mixer window

The Audio Mixer window contains the following areas:

Audio Mixer asset: Contains all groups and snapshots as subassets.

Hierarchy view: Shows the entire mixing hierarchy of groups within the Audio Mixer.

Mixer Views: Lists cached visibility settings for the mixer. Each view shows only a subset of the hierarchy in the main mixer window.

Snapshots: Lists all snapshots within the Audio Mixer asset. Snapshots capture the state of every parameter setting in an Audio Mixer, and you can transition between them at runtime.

Output Audio Mixer: Routes this Audio Mixer's signal into a group of another Audio Mixer. Use this property to define the output group.

AudioGroup strip view: Shows an overview of an AudioGroup, including the volume unit (VU) levels, attenuation (volume) settings, **Mute**, **Solo**, and **Effect Bypass** settings, and the list of DSP effects within the group.

Edit in Play Mode toggle: Lets you edit the Audio Mixer during Play mode or prevent edits, so the application runtime controls the state of the Audio Mixer.

Exposed Parameters: Lists the exposed parameters and their string names. You can expose any parameter within an Audio Mixer to a script through a string name.



Audio Mixer Inspector window

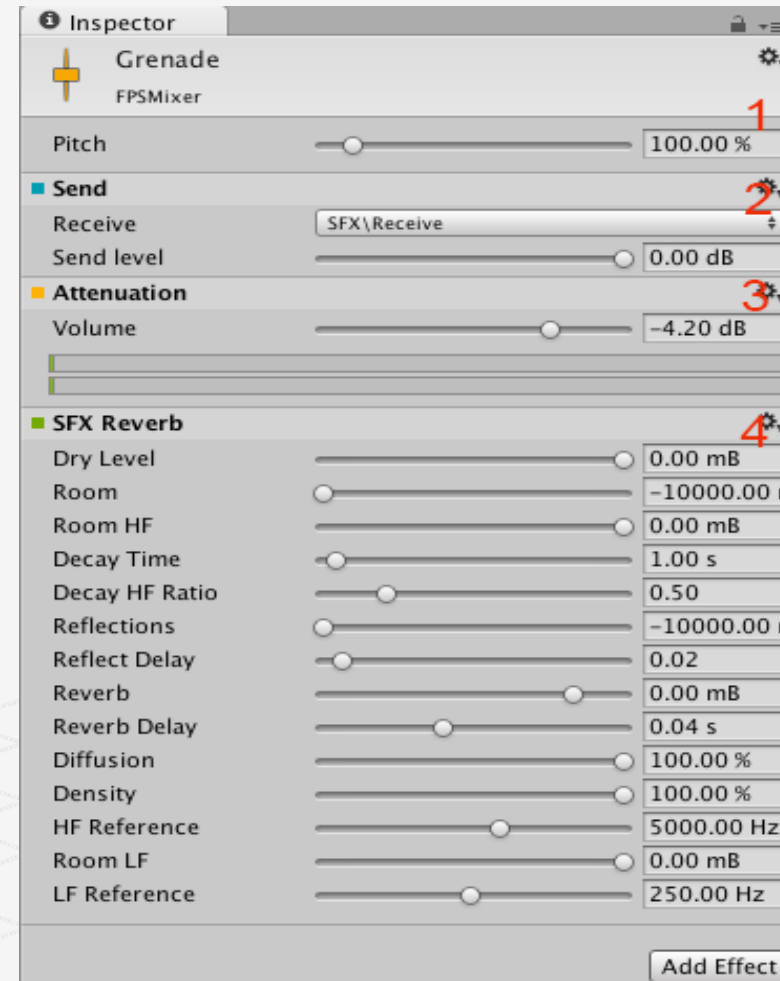
The Audio Mixer Inspector displays the configurable components of a group:

Pitch and **Ducking** settings: Appear at the top of every group.

Send: An example send effect, positioned before the point where Unity applies attenuation.

Attenuation: Sets the volume for a group. You can apply attenuation anywhere in the effect stack. The VU meter shows the volume level at that point in the stack, which differs from the VU meters in the Audio Mixer view that show the level of the signal as it leaves the group.

SFX Reverb: An example effect with parameters, in this case a reverb. To expose a parameter, right-click it, and select the option.

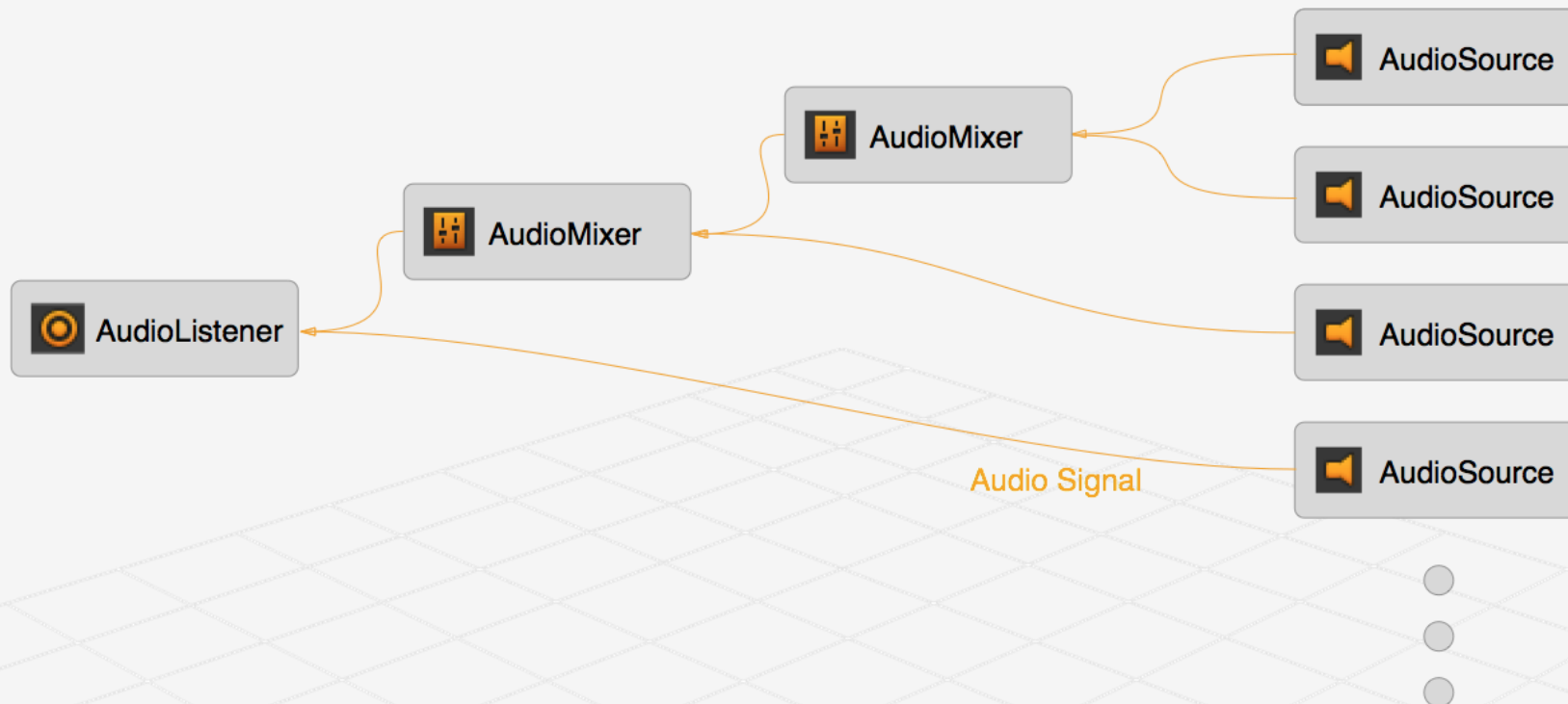


Routing and mixing

Audio routing is the process of taking input audio signals and producing one or more output signals. A signal is a continuous stream of digital audio data, which you can break down into digital audio channels, such as stereo or 5.1 (six channels).

You can use the Audio Mixer to perform important audio processing work on these signals. Examples include mixing, applying effects, and attenuation.

With the exception of sends and returns, the Audio Mixer contains groups that accept any number of input signals, mix those signals, and produce exactly one output.



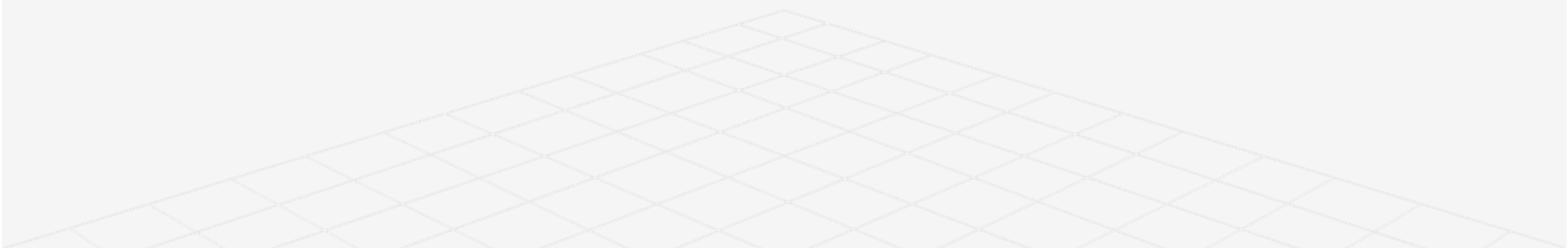
The relationship between different audio entities

One Audio Mixer receives input signals from two different Audio Sources. A second Audio Mixer receives input signals from the first Audio Mixer and a third Audio Source. An Audio Listener receives input signals from the second Audio Mixer and a fourth Audio Source.

In audio processing, this routing and mixing usually happens independently of the scene graph hierarchy because audio behaves differently from the objects and concepts shown in the scene. You interact with it differently too.

Without an Audio Mixer, the signal that each Audio Source produces (through audio clips, for example) routes directly to the Audio Listener, where Unity mixes all the audio signals at a single point. This mixing happens independently of the scene graph, regardless of where your Audio Sources are in the scene.

An Audio Mixer sits between the Audio Source and the Audio Listener in the audio signal processing chain. It takes the output signal from the Audio Source and performs the routing and mixing operations you want, until Unity outputs all audio to the Audio Listener and plays it through the speakers.



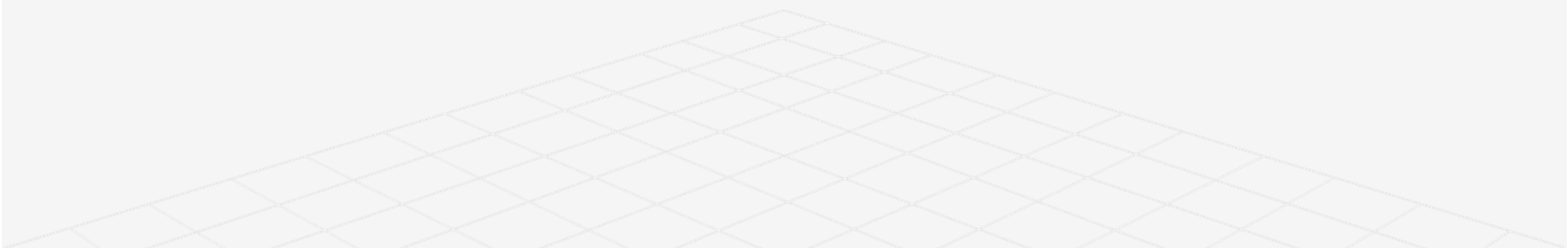
Benefits of routing and mixing

With mixing and routing, you can categorize the audio in your application into the categories you want.

After you mix sound into these categories, you can apply effects and other operations to each category as a whole.

This is powerful for applying logic changes to sound categories.

You also can tweak aspects of the mix to perform what's known as mastering of the entire soundscape dynamically at runtime.



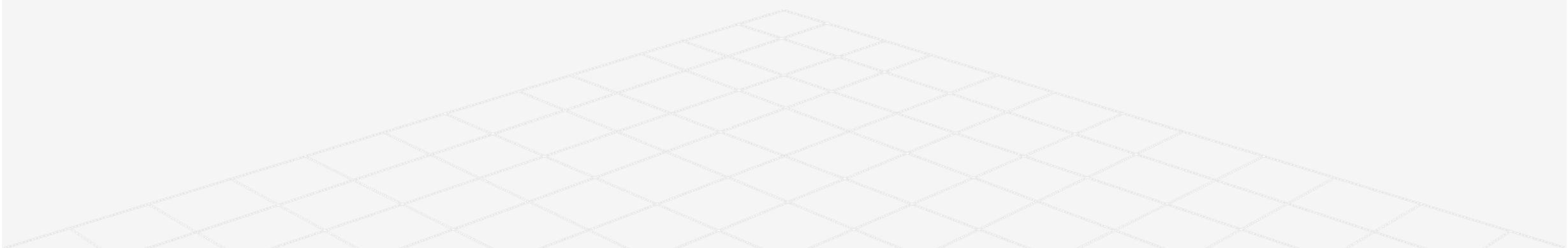
Relationship to 3D spatial attenuation

Some sound concepts relate to the scene graph and the 3D world.

The most obvious examples are attenuation based on 3D distance, relative speed to the Audio Listener, and environmental reverb effects.

Because these operations relate to the scene rather than to the categories of sounds in an Audio Mixer, Unity applies the effects at the Audio Source, before the signal enters an Audio Mixer.

For example, Unity applies the distance-based attenuation for an Audio Source to the signal before the signal leaves the Audio Source and routes into an Audio Mixer.



Sound categories

With an Audio Mixer, you can categorize types of sounds and apply operations to those categories.

This categorization is important. Without it, every sound plays back equally and without any mixing, and the soundscape becomes difficult to distinguish.

With techniques such as ducking (automatically lowering the volume of one category when another plays), categories of sounds can influence each other, which adds richness to the mix.

You might want to perform the following operations on a category:

- Apply attenuation to a group of ambient sounds.
- Trigger a low-pass filter on all the foley sounds in your application to simulate being underwater.
- Attenuate all sounds except the menu music and interaction sounds.
- Reduce the volume of all the gun and explosion sounds so the user can hear a non-player character (NPC) speaking.

Example

The following is an example of one such categorization:

All sounds route into the **Master** group.

The Master group contains categories for music, menu sounds, and all application sounds.

The application sounds group breaks down into dialogue from NPCs, environmental sounds from ambient sources, and other foley sounds such as gunshots and footsteps.

These categories break down further as required.

The following image shows the category hierarchy of this layout:



Moods and themes of the mix

You can also use mixing and routing to create the immersion you want.

For example, you can apply reverb to all the application sounds and attenuate the music to create the feeling of being in a cave.

You can use the Audio Mixer to create moods in your application.

With techniques such as snapshots and multiple mixers, your application can transition its mood and evoke the feelings you want the user to experience, which strengthens immersion.

The global mix

The Audio Mixer controls the overall mix of all the sounds in your application.

An Audio Mixer controls the global mix.

It's the static, persistent mix that sound instances route through.

In other words, an Audio Mixer is always present throughout the lifetime of a scene.

Unity creates and destroys sound instances as your application runs, and those instances play through these global Audio Mixers.

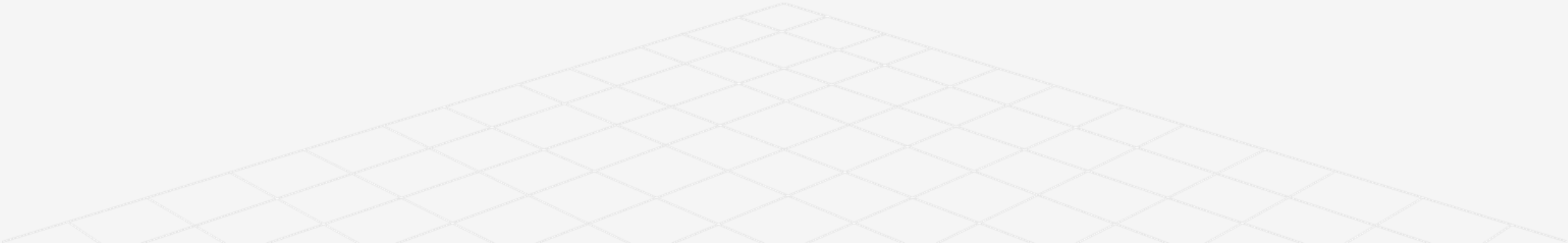
Snapshots

Snapshots capture the state of an Audio Mixer and transition between those states as your application runs. Use snapshots to define moods or themes for the mix and change those moods as the user progresses through your application.

Snapshots capture the values of all the parameters in the Audio Mixer:

- Volume
- Pitch
- Send level
- Wet mix level
- Effect parameters

Combine snapshots with application logic to change many aspects of the soundscape.



Audio Mixer window

The window displays the Audio Mixer which is basically a tree of Audio Mixer Groups.

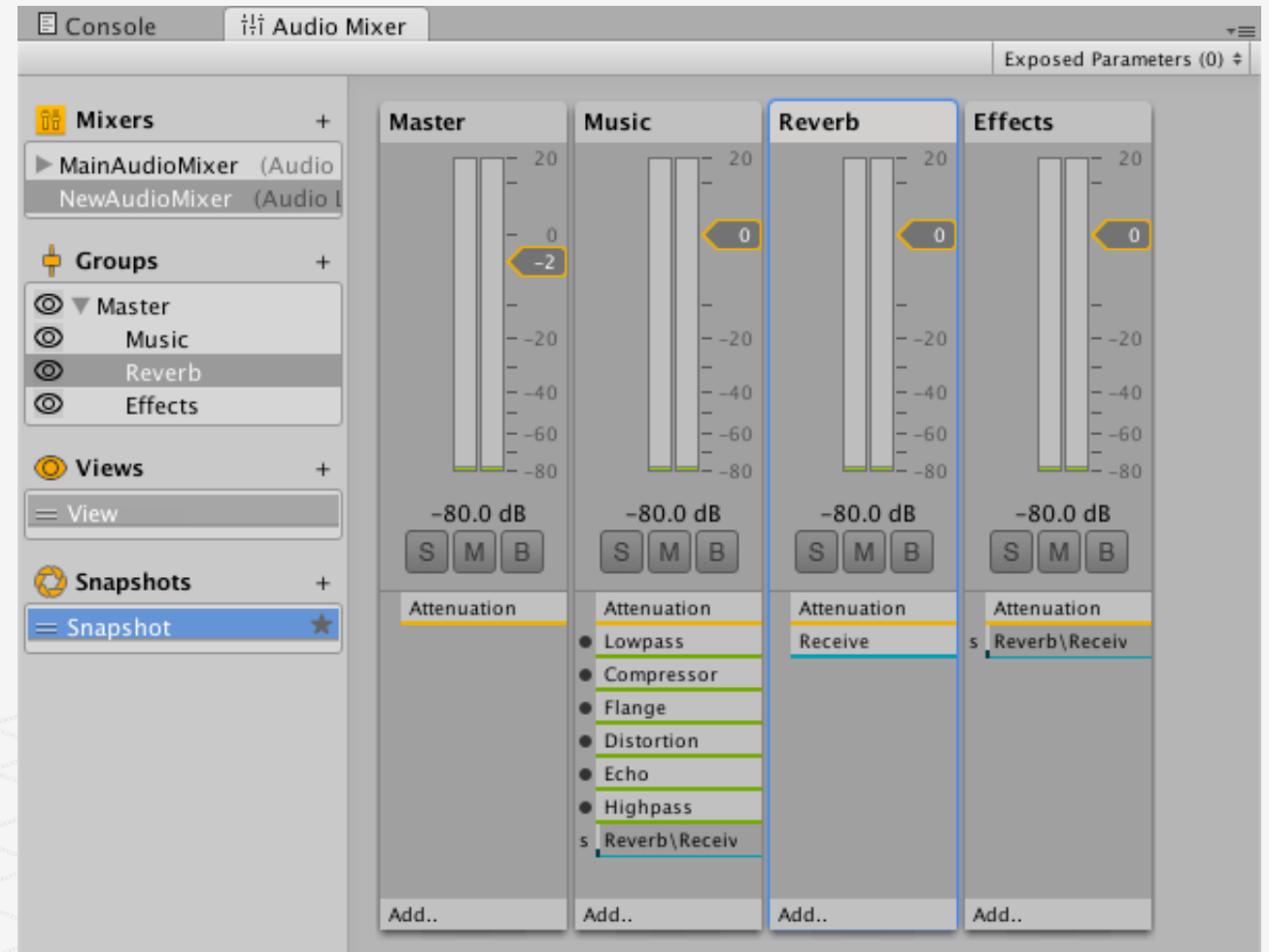
An Audio Mixer group (bus) is a mix of audio signals processed through a signal chain which allows you to control volume attenuation and pitch correction.

It allows you to insert effects that process the audio signal and change the parameters of the effects.

There's also a send and return mechanism to pass the results from one bus to another.

An Audio Mixer is an asset. You can create one or more Audio Mixers and have more than one active at any time. An Audio Mixer always contains a master group. Other groups can then be added to define the structure of the mixer.

Note: The Web platform only partially supports Audio Mixers. For more information on how audio is used in the Web platform, refer to [Audio in Web](#).



How it works?

You route the output of an [Audio Source](#) to a group within an Audio Mixer.

The effects will then be applied to that signal.

The output of an Audio Mixer can be routed into any other group in any other Audio Mixer in a scene enabling you to chain up a number of Audio Mixers in a scene to produce complex routing, effect processing and snapshot applying.

Snapshots

You can capture the settings of all the parameters in a group as a snapshot. If you create a list of snapshots you can then transition between them in gameplay to create different moods or themes.

Ducking

Ducking allows you to alter the effect of one group based on what is happening in another group. An example might be to reduce the background ambient noise while something else is happening.

Views

The Views section allows you to enable and disable the visibility of certain groups within a mixer and set them as a view.

All groups are visible by default, but you can set up a view to only show the groups you want to edit at specific times. This can be useful to reduce clutter.

Native audio plug-in SDK

The Unity native audio plug-in SDK lets you develop custom audio plug-ins for Unity. You can use this SDK to extend the audio capabilities of Unity and create advanced audio processing solutions tailored to your project's needs. Examples of custom audio plug-ins you can create include audio effects and audio spatializers.

The native audio plug-in system consists of two parts:

The native audio Digital Signal Processing (DSP) plug-in

The managed graphical user interface (GUI)

Topic	Description
Develop a native audio DSP plug-in for Unity	Learn how to create your own native DSP plug-in.
Customize the Unity GUI for your audio plug-in	Learn how to customize the GUI of your audio plug-in.
Import your audio plug-in and GUI to Unity	Learn how to prepare your plug-in and GUI for Unity and import them.
Example plug-ins	Example DSP plug-ins with and without GUI customization.
Audio Spatializer SDK	Change the way your application transmits audio from an audio source into the surrounding space.

Audio playlist randomization

Use an Audio Random Container asset to set up a playlist that you can play back in different ways.

You can use it for most sound effect use cases, such as footsteps, impacts, weapons, and props.

Topic	Description
Audio Random Container reference	Understand the user interface of the Audio Random Container.
Audio Random Container fundamentals	Understand the audio concepts you need to use the Audio Random Container.
Create a randomized playlist with the Audio Random Container	Create and use a randomized audio playlist.

Ambisonic audio

Understand the options for using ambisonic audio in Unity.

Ambisonics are a type of audio that provide a representation of sound that can completely surround a listener.

Topic	Description
Introduction to ambisonic audio	Understand how ambisonic audio works in Unity and what formats Unity supports.
Use ambisonic audio in your project	Select an ambisonic audio decoder, import ambisonic audio clips, and play them.
Introduction to ambisonic audio decoders	Learn about ambisonic audio decoders.
Develop an ambisonic audio decoder	Instructions for implementing an ambisonic decoder plug-in and registering it in Project Settings.

Audio Reference

Topic	Description
Audio Clip	Reference information about the AudioClip class.
Audio Listener	Reference information about the AudioListener class.
Audio Source	Reference information about the AudioSource class.
Audio Mixer window reference	Reference information about the Unity AudioManager class and its UI.
Audio filters	Reference information about the various audio filters in Unity.
Audio effects	Reference information about the various audio effects in Unity.
Audio Reverb Zone	Reference information about the AudioReverbZone class.
Microphone	Reference information about the Microphone class.
Audio Settings	Reference information about the AudioSettings class.